

Data Science →

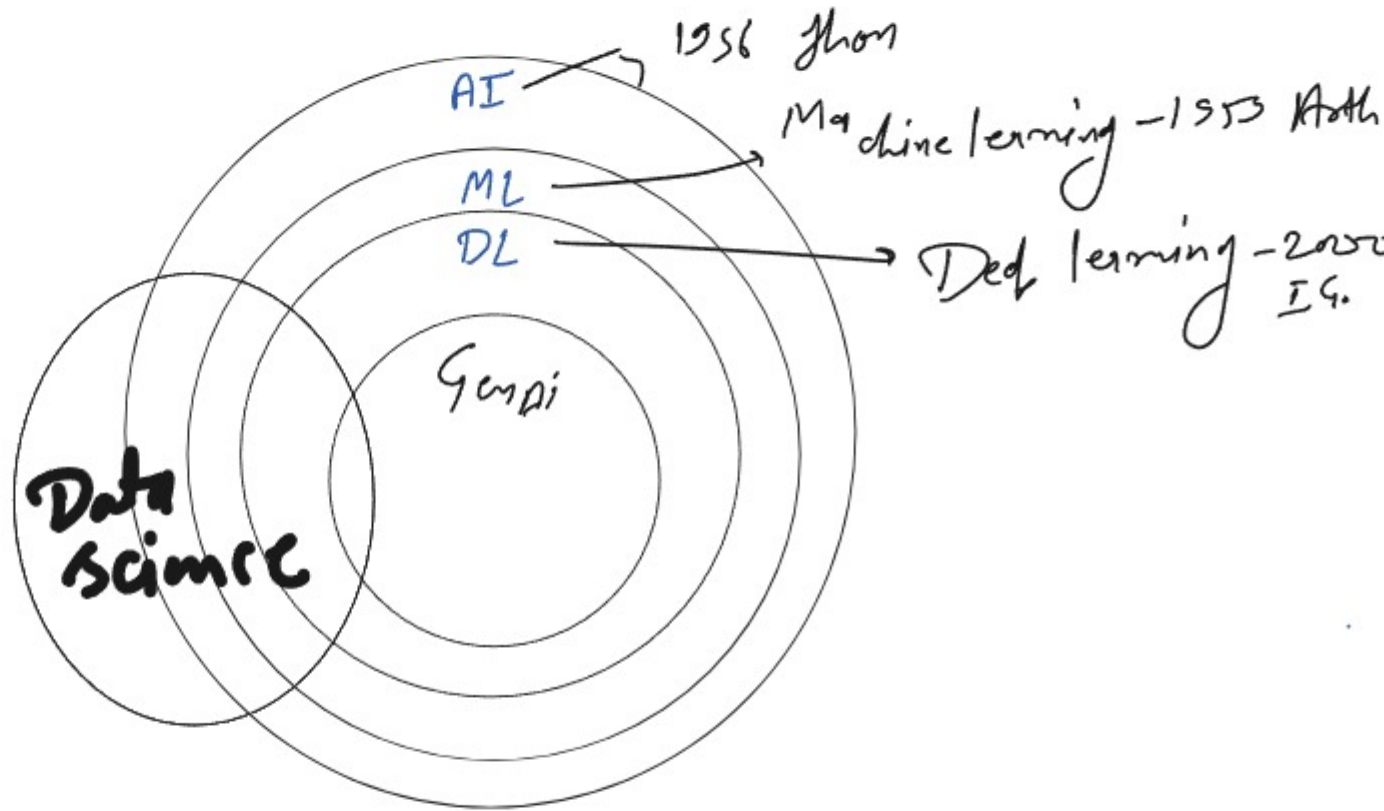
Data

data

Science

study

Study of data that is called as data science



Machine learning

Machine

Learning

① Sub.

② Unsup

③ Rein

Machine Learning

Machine

Learning

types

① Supervised $\Rightarrow$ lab 4

② Unsupervised

③ Reinforcement

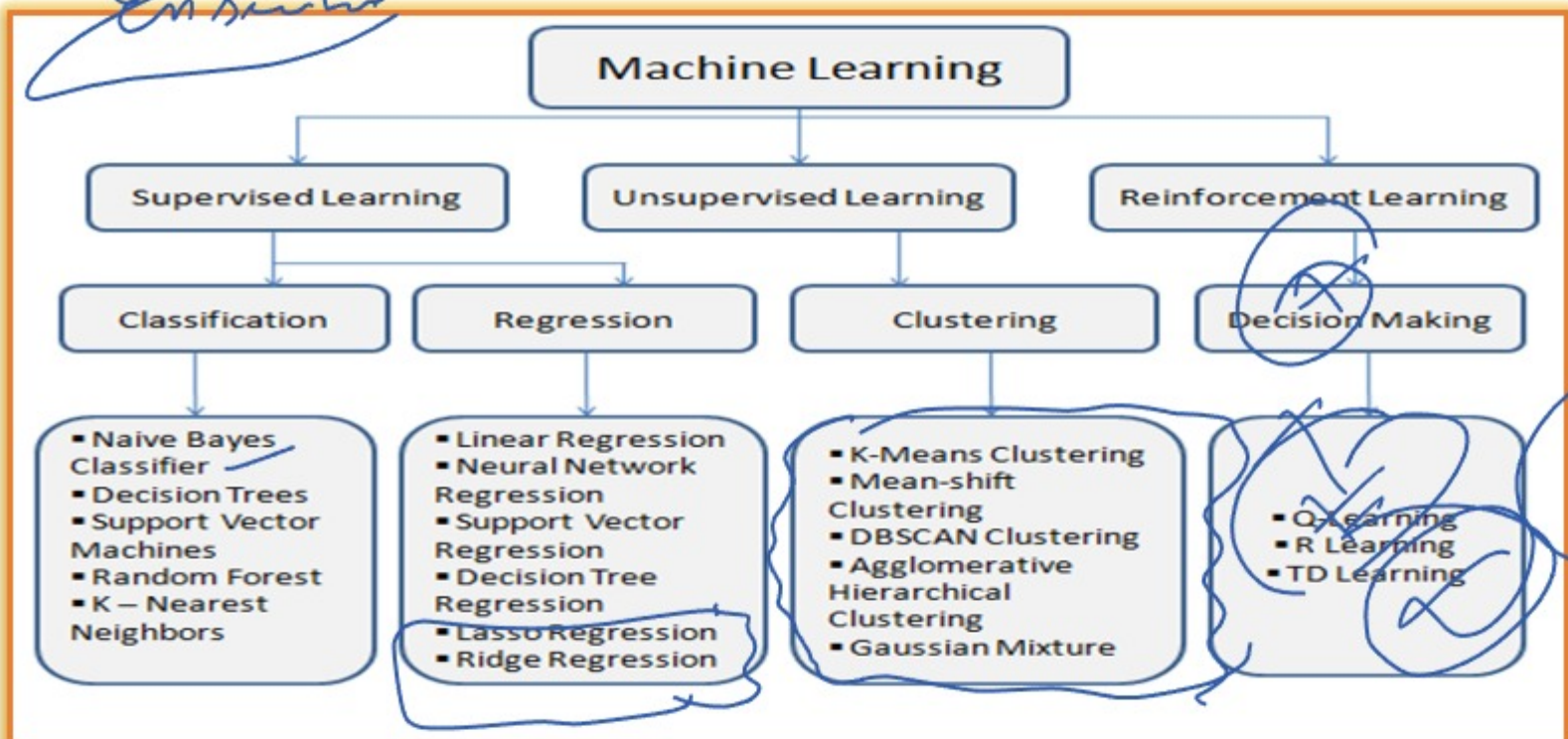
④ Semi supervised

data collect

Learning $\rightarrow$ Preference, $\Sigma P \Delta$

data

Ensemble



Classification
Positive / neg
Total / class
Yes / no

Represent Ada

Ensemble

Row

x_g

x_g

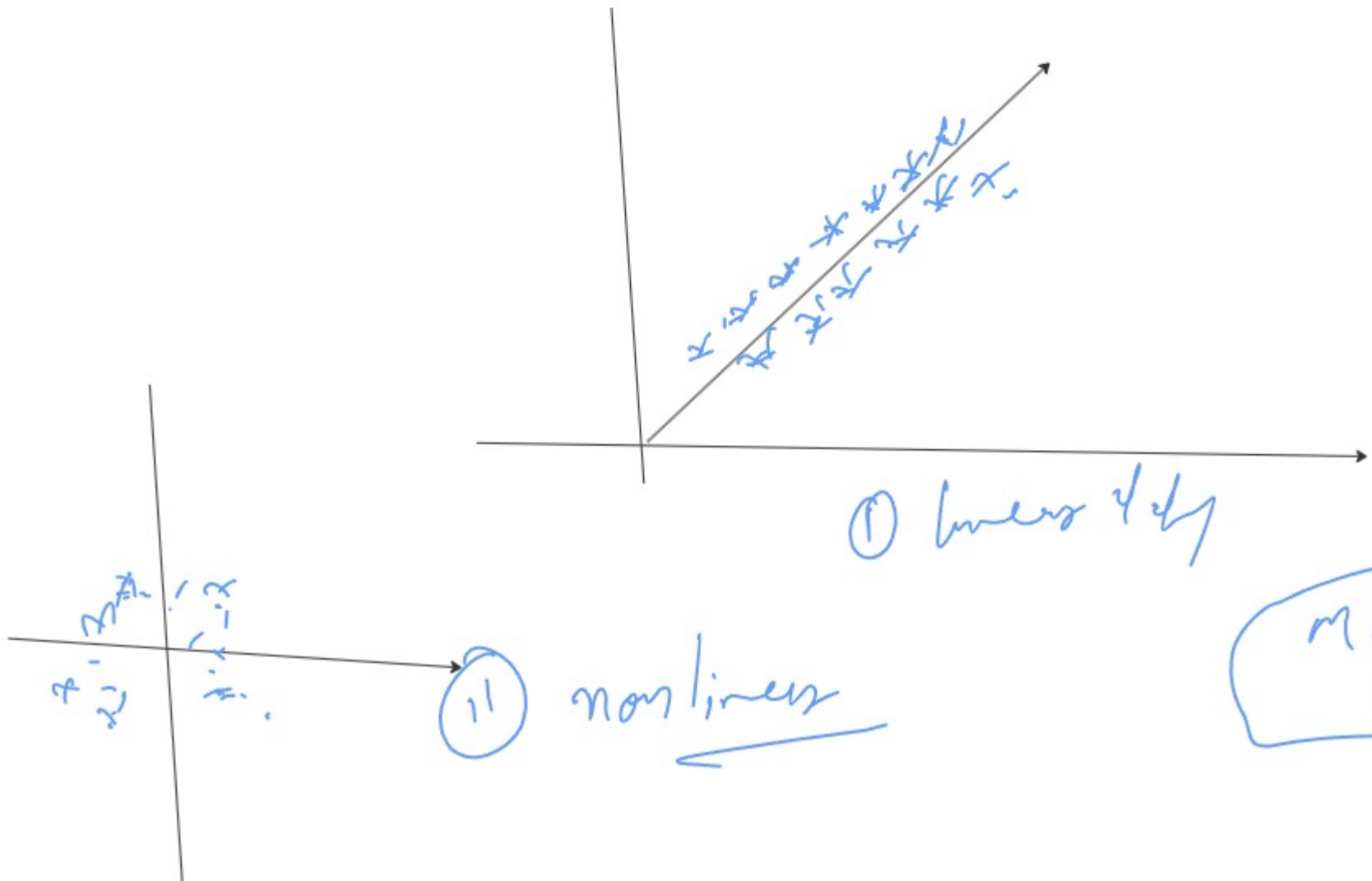
x_g

Bagging
Boosting
Stacking

100%

Linear Regression
 # supervised
 # Regression
 # label

Linear and non linear;



ML-PL
 ML

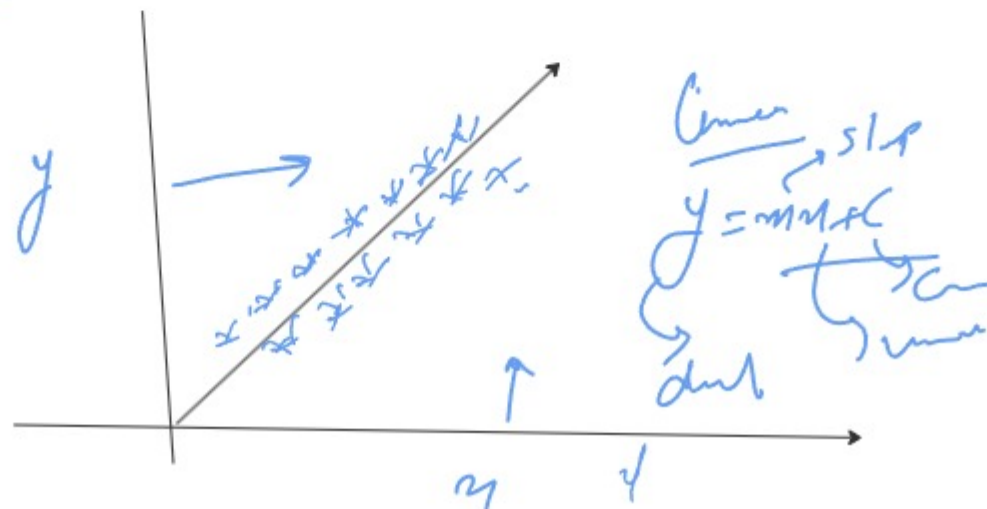
Linear Regression

$$y = mx + c$$

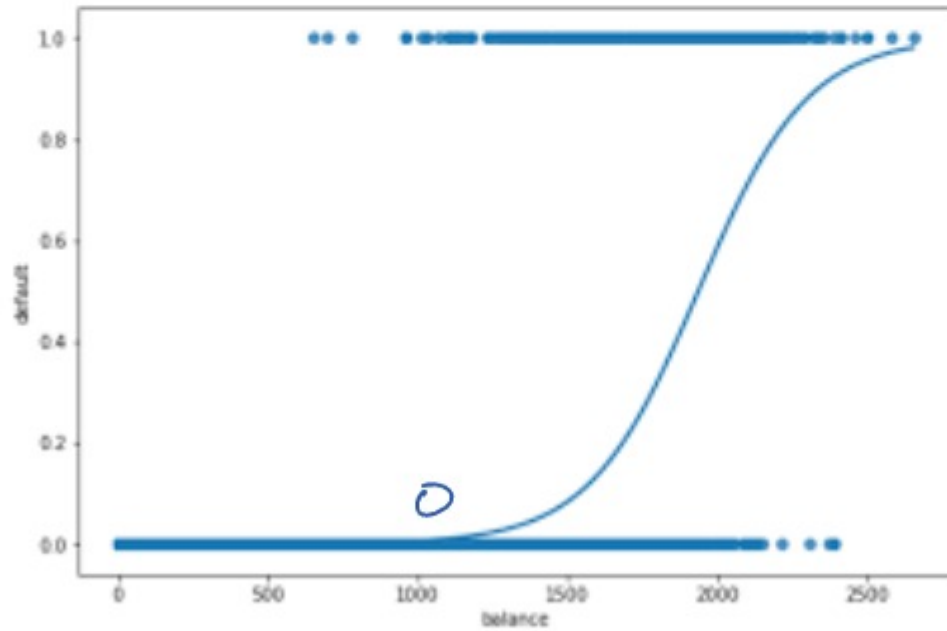
y → dependant variable
 x → independent variable
 m → slope
 c → constant

Bike
Price

Brand name, age, kms, owner, Power



logistic Regression / ANW
 # sub. , # label
 Yes / no (1/0)
 Sigmoid



1 Sigmoid
 (0/1)
 output
 true / false

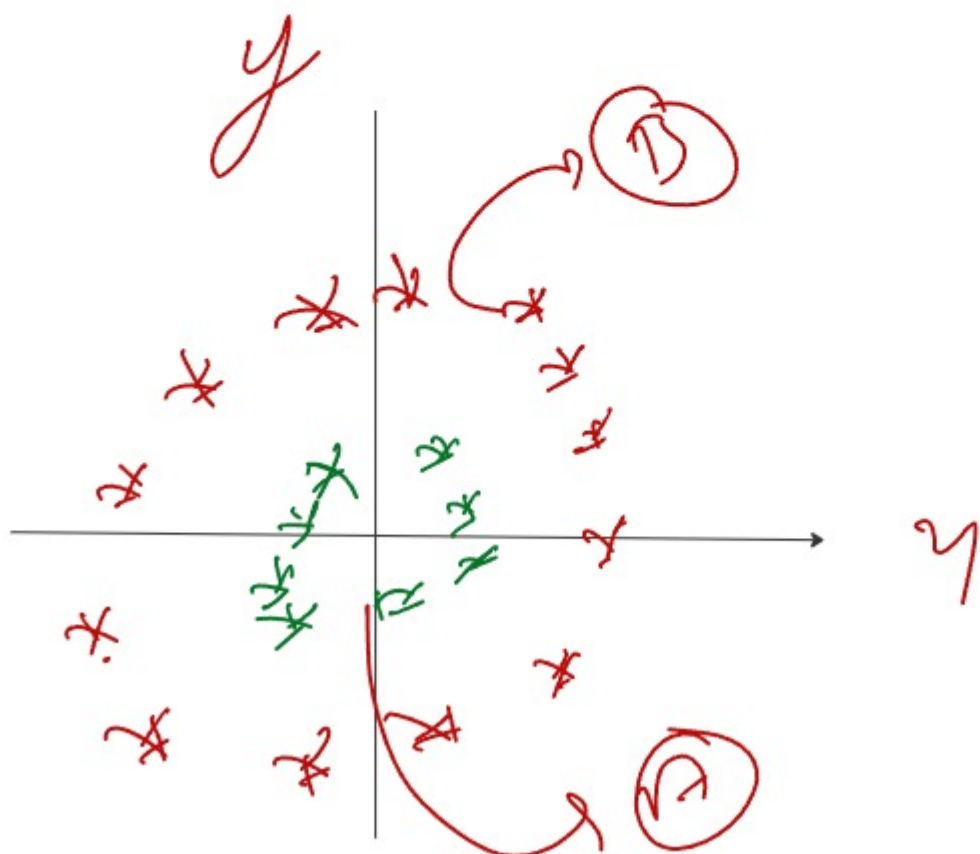
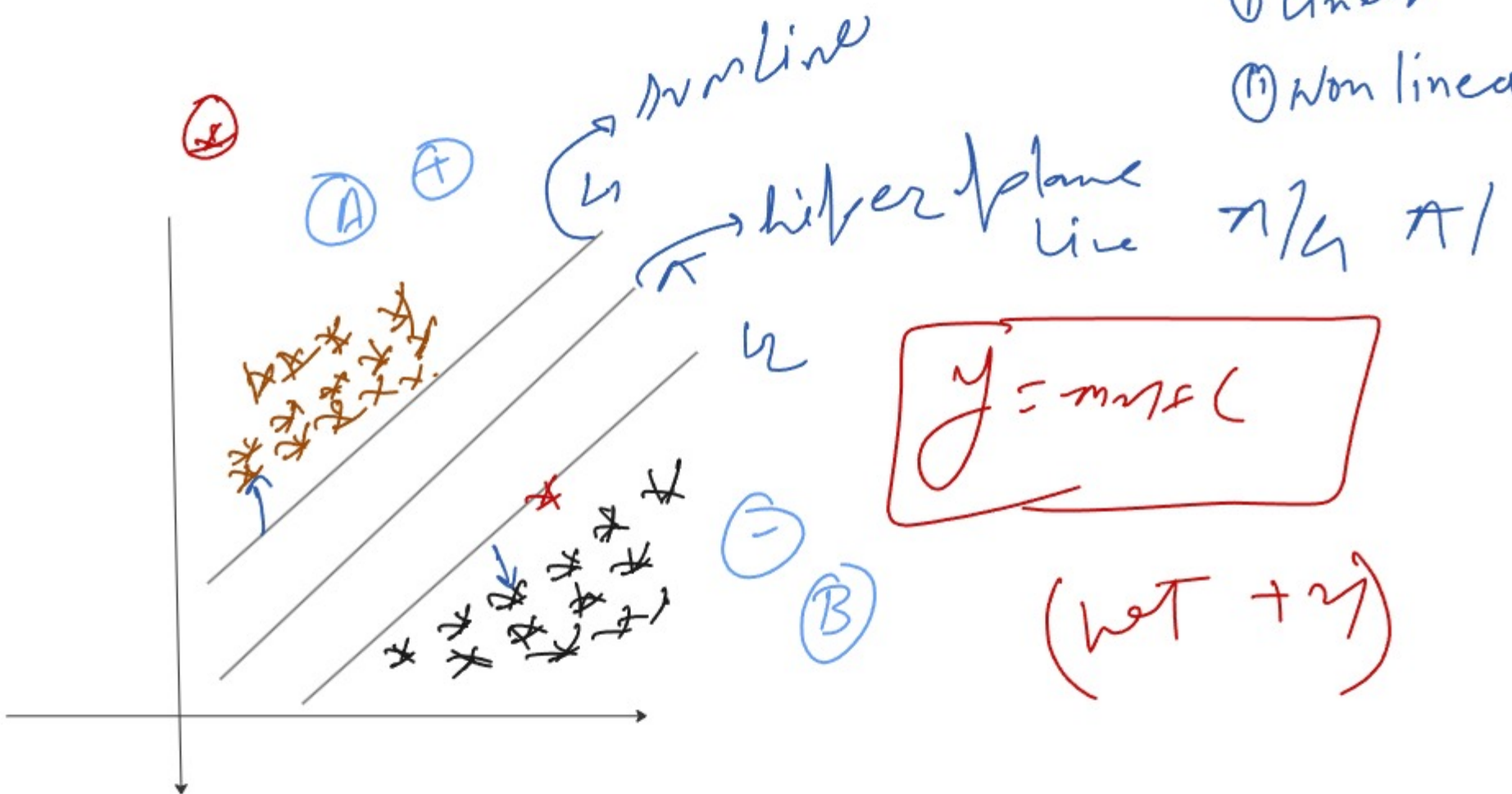
formula = $\sigma(z) = \frac{1}{1 + e^{-z}}$
 ortho.
 $e = 2.718$

SVM
 (Support vector machine)
 # supervised
 # label
 # classification (mostly)
 &
 Regression

SVM R & SVM C

types

- ① Linear
- ② Non linear



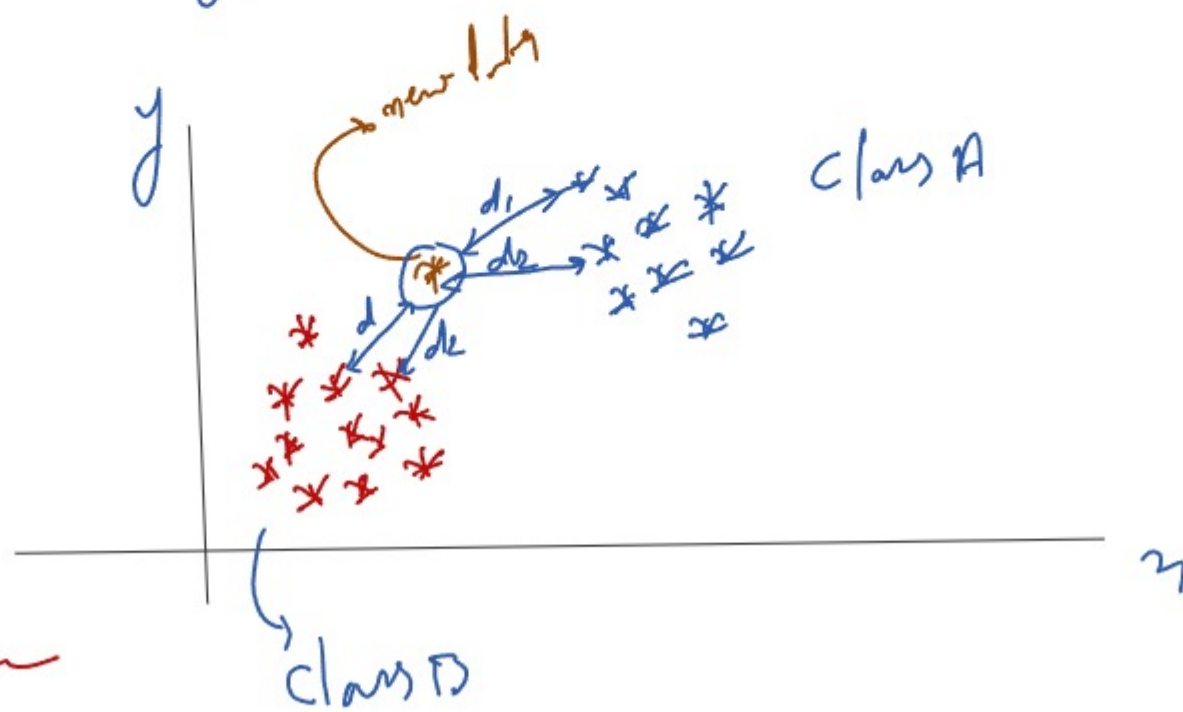
KNN (K-Nearest Neighbor)

#1 Sub.

K = ?

Classification (mainly)

Regression



$$d(x, y) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

2. Manhattan distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. Minkowski distance

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{1/p}$$

$$\begin{matrix} p=2 \\ p=1 \end{matrix}$$

Recommendation
Span detection

Advantage —

Simple to use
no training step
Parameter

dis

slow (T) detected (T)
overfitting

classification report

classification

A confusion matrix for a spam classification task. The matrix is a 2x2 grid. The columns are labeled 'Predicted' and the rows are labeled 'Actual'. The top-left cell (True Positive) is green and contains '600 (True positive)'. The top-right cell (False Negative) is red and contains '300 (False negative)'. The bottom-left cell (False Positive) is red and contains '100 (False positive)'. The bottom-right cell (True Negative) is green and contains '9000 (True negative)'. Handwritten red annotations include: 'Spam' and 'Non-spam' above the columns; 'Spam' and 'Non-spam' to the left of the rows; 'Correct' above the True Positive cell; 'Incorrect' above the False Negative cell; 'help to doctor' with an arrow pointing to the False Positive cell; and various checkmarks and arrows indicating model performance.

	Predicted	
	Spam	Non-spam
Actual	600 (True positive)	300 (False negative)
Non-spam	100 (False positive)	9000 (True negative)

Performance Metrics /
Model Evaluation Techniques

classification

Accuracy

Precision

Recall

f1 score

Roc - Auc

Regression metrics

MAE

MSE

RMSE

HUBBER loss

Accuracy

$$= \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$= \frac{TP}{TP + FP}$$

Recall

$$= \frac{TP}{TP + FN}$$

F1 Score

$$\hookrightarrow = 2 \times \frac{Pr \times Re}{Pr + Re}$$

Regression metrics

① MSE

② MAE

③ RMSE

④ Huber loss

① MSE (Mean Squared Error)

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

② MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

RMSE (Root Mean Squared Error)

$$RMSE = \sqrt{MSE}$$

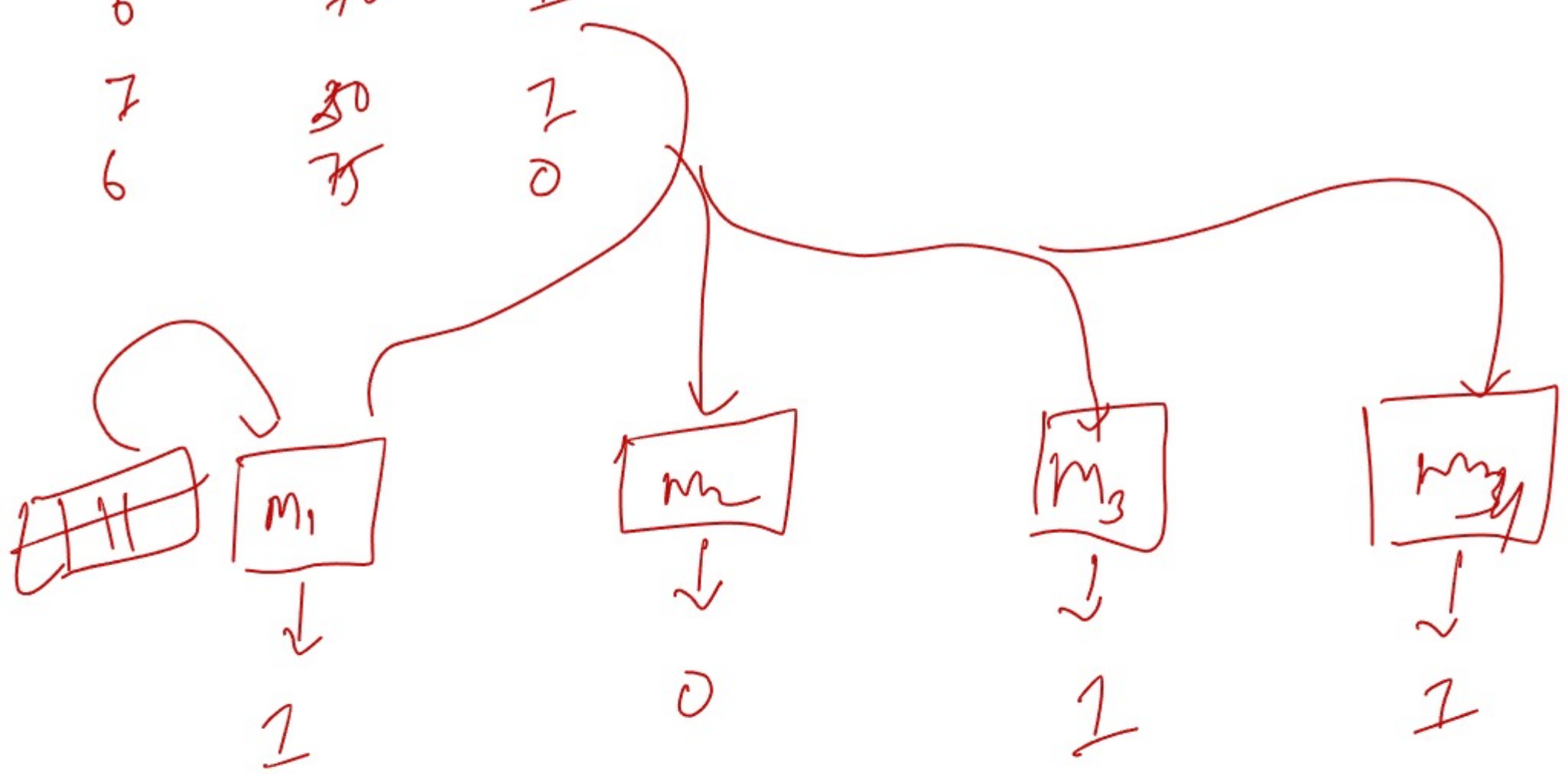
Huber loss = $MSE + MAE$

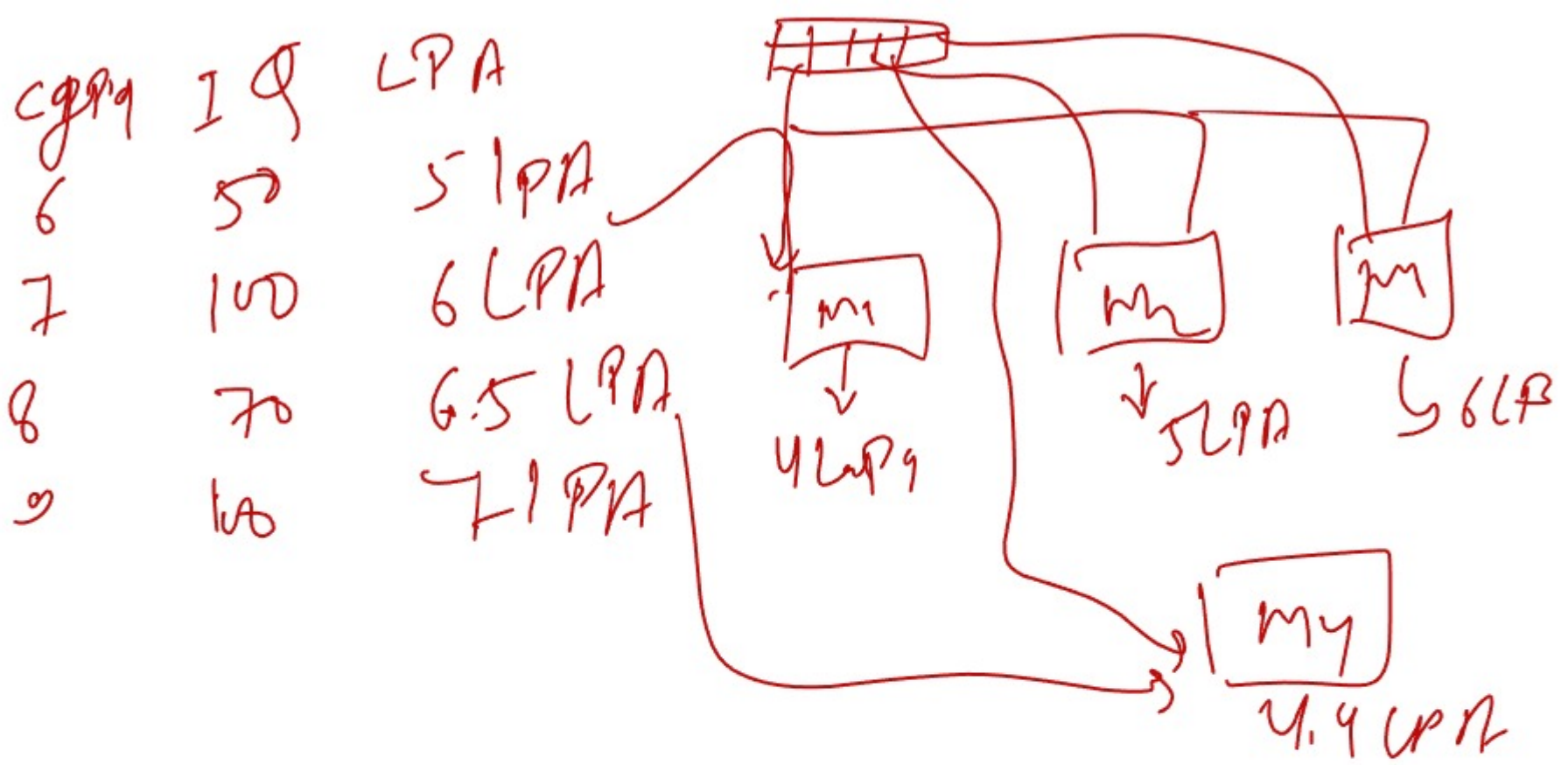
Ensemble Learning



2400 kg
2350 kg

Cgpa	IQ	Place
9	50	0
8	70	1
7	80	1
6	75	0





$$\frac{4 + 5 + 6 + 4.5}{4} =$$

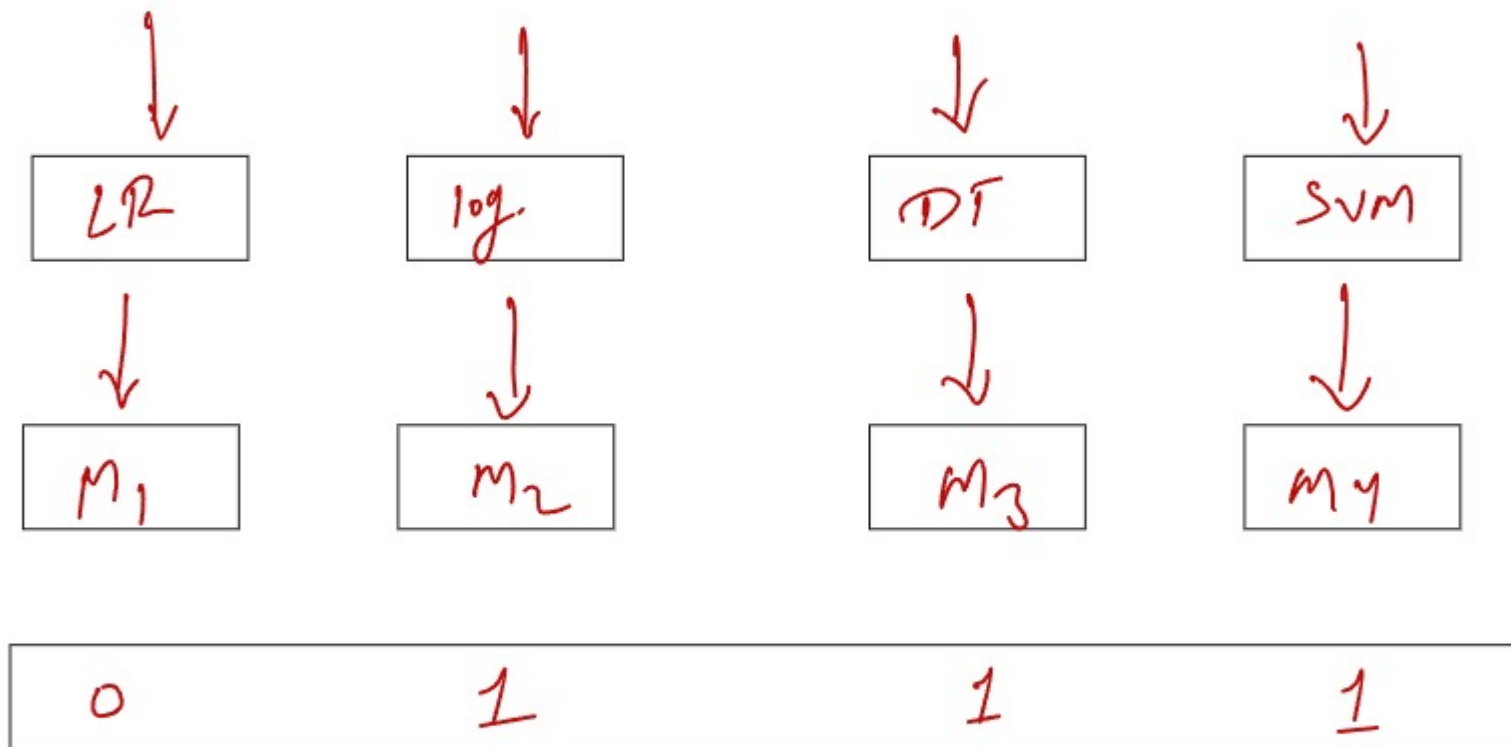
- (i) Bagging
 - (ii) Boosting
 - (iii) Voting
 - (iv) Stacking
- Algo Same $\Rightarrow$ Same model
- Algo diff $\Rightarrow$ model diff.

from sklearn ensemble

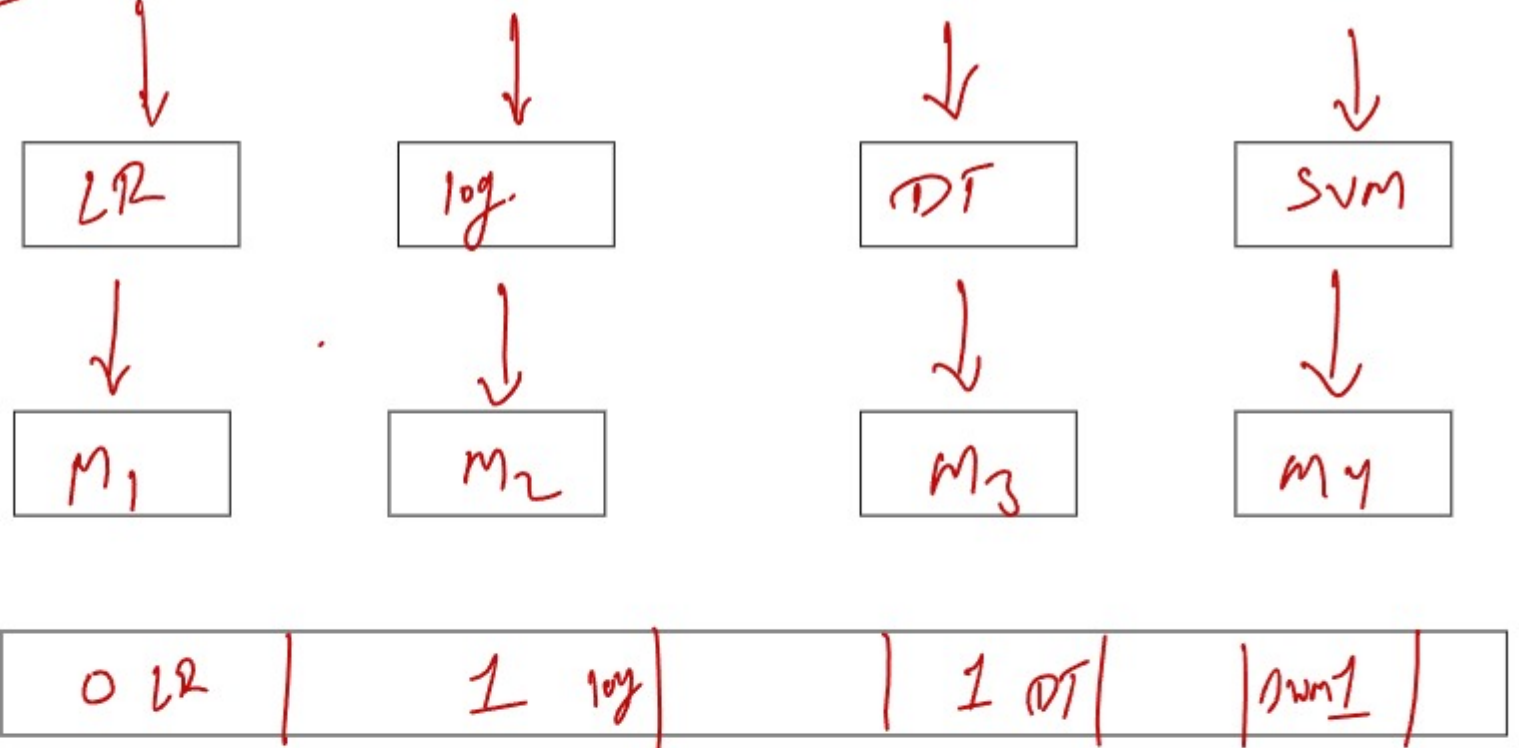
in voting

Voting Algorithm.

from sklearn.ensemble import voting classifier , stacking



Stacking



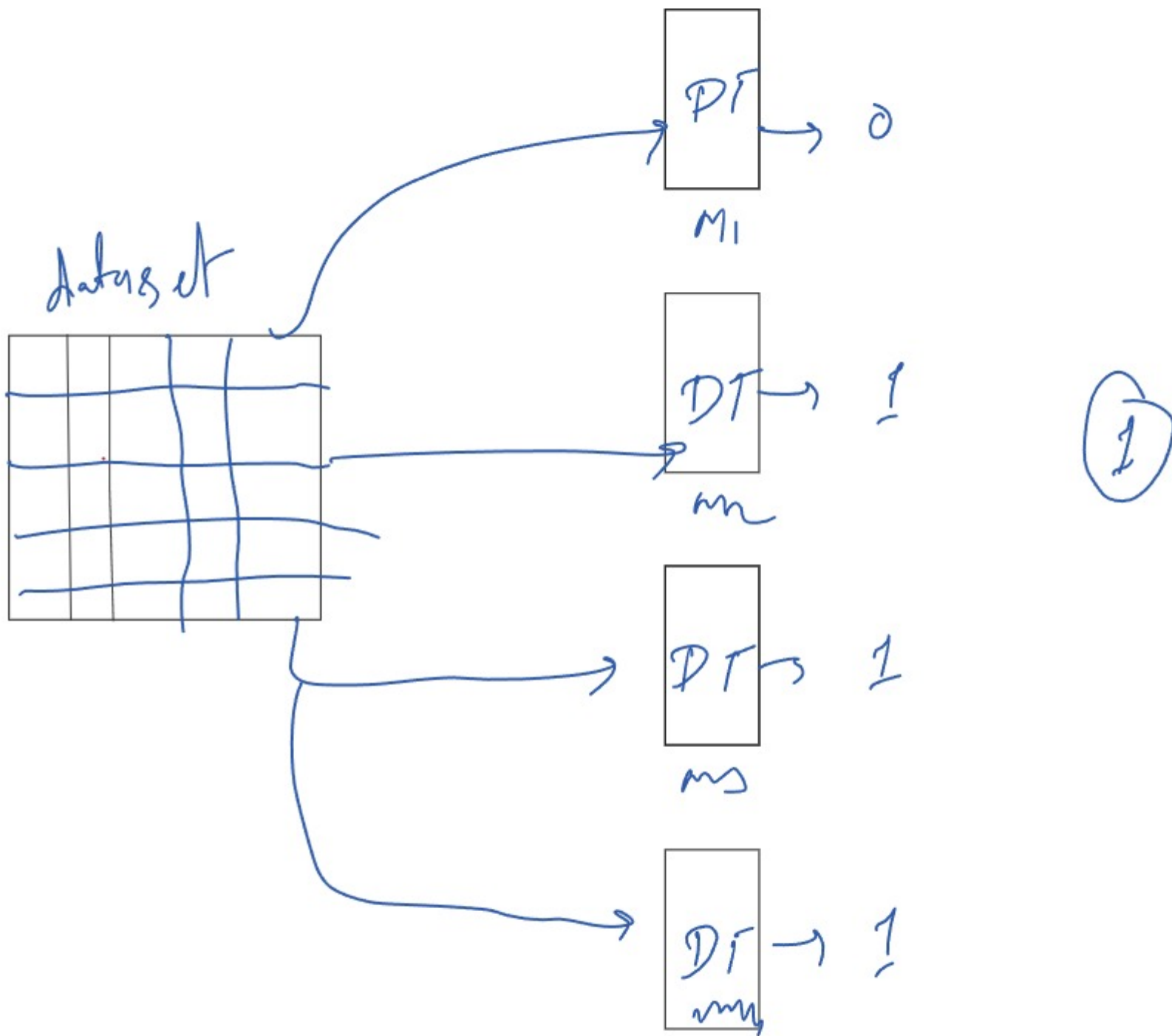
Final result



Alg.	Pred
LR	0
log	1
DT	1
SVM	1

Bagging & Boosting
Algo - Same, Model - Same

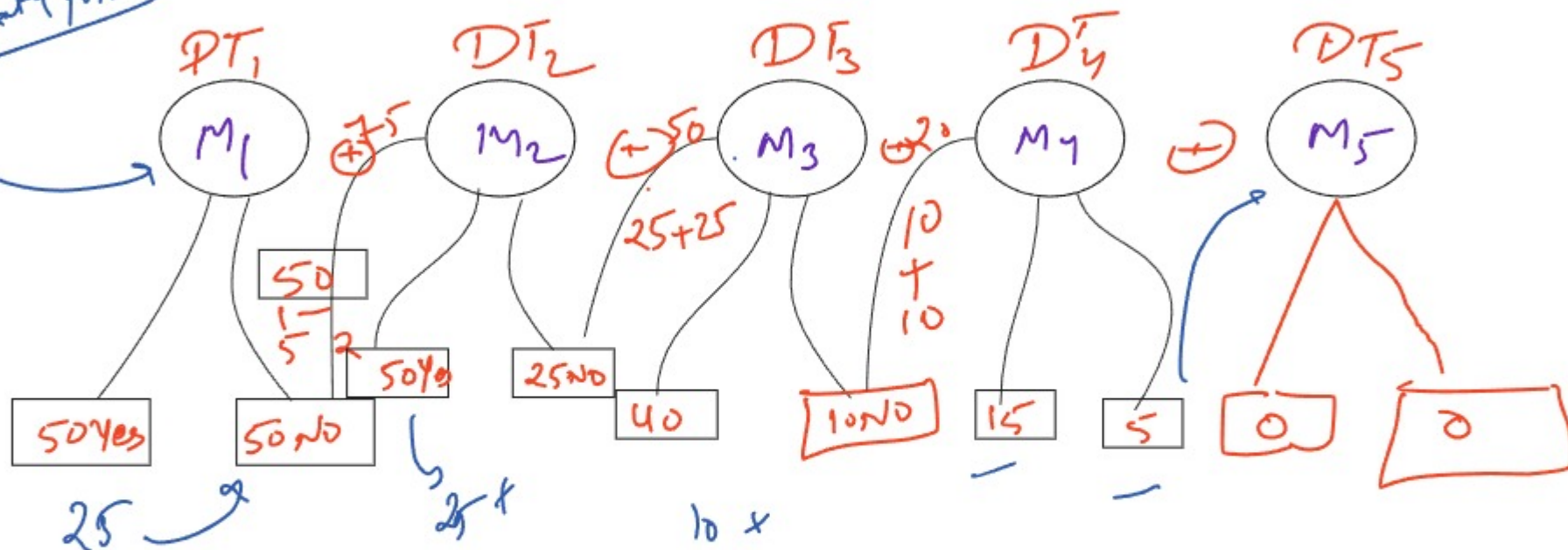
Bagging Ensemble Learning
Algo Same



② Random forest

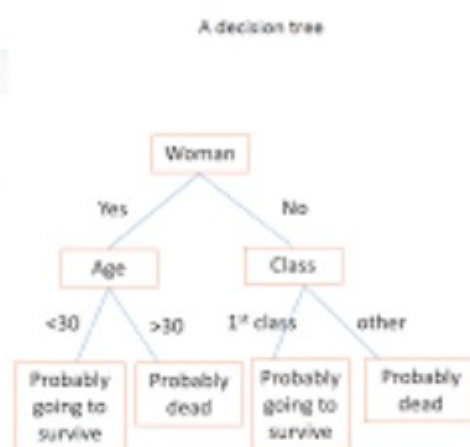
Boosting $\rightarrow$ Algo.

100 data point



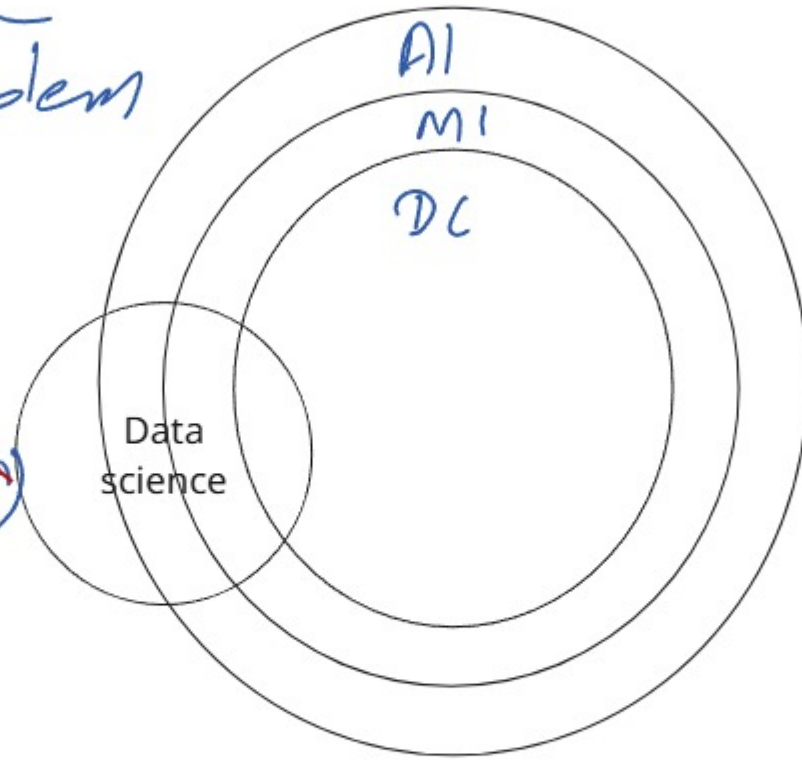
Algo

AdaBoost
x GBoost
Gradient Boosting Algo



Deep learning

Subset of ML
 # Complex problems
 solve
 # dataset (↑)
Accuracy (↑)



ML
 dataset (↑)
 Accuracy (↓)
Score (↓)

Algo of DL

- (i) ANN (Artificial Neural Network)
- (ii) CNN (Convolutional Neural Network)
- (iii) RNN (Recurrent Neural Network)

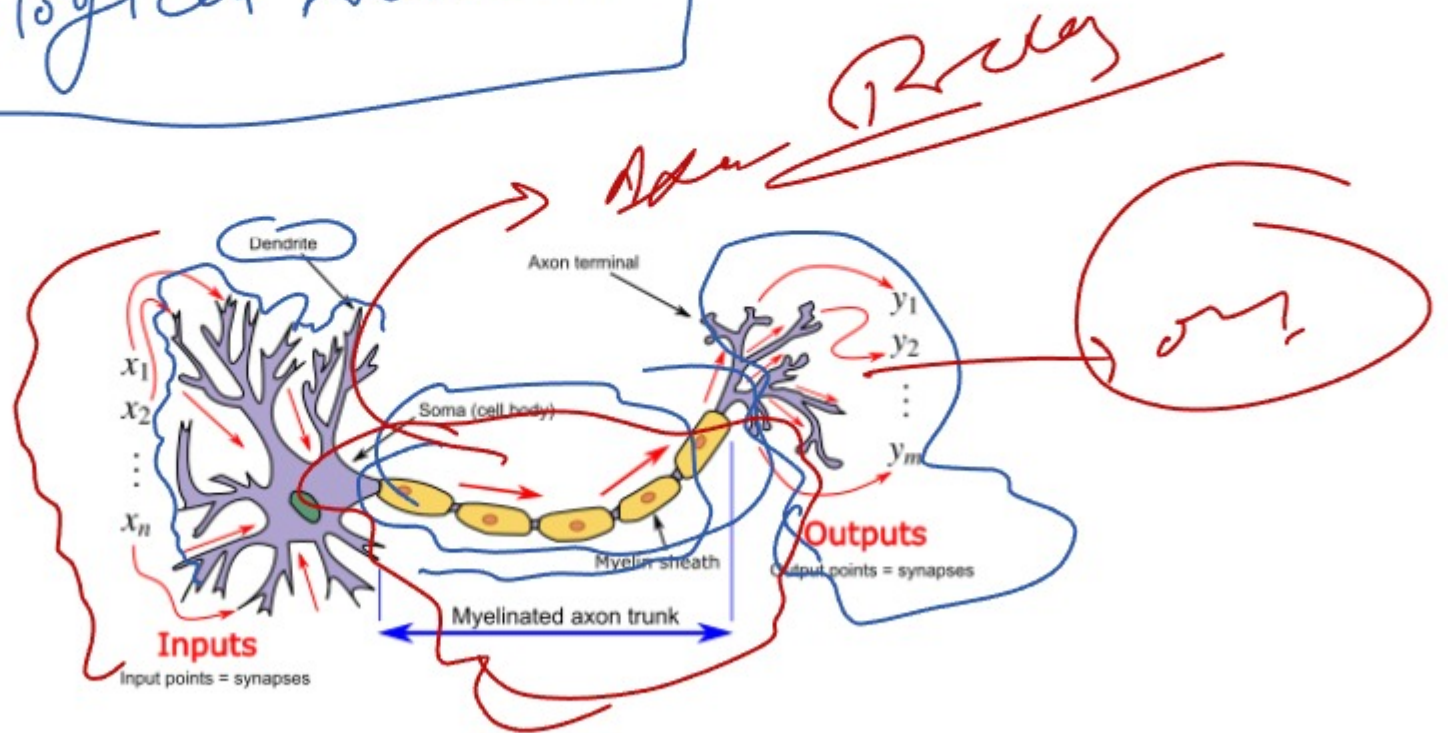
LSM
 GRU

ANN

Artificial Neural Network

Biological Neuron

Input



ANN_____

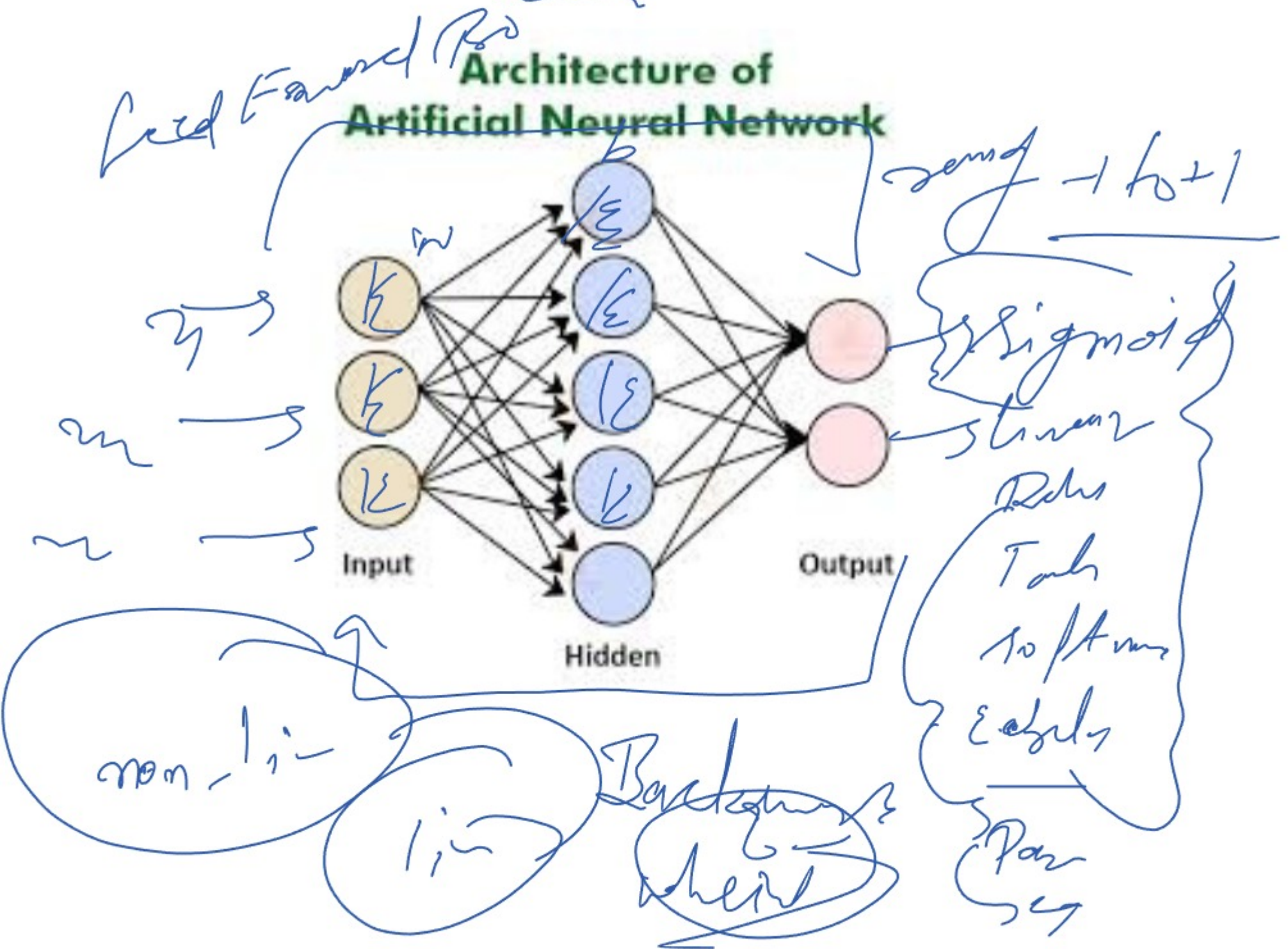
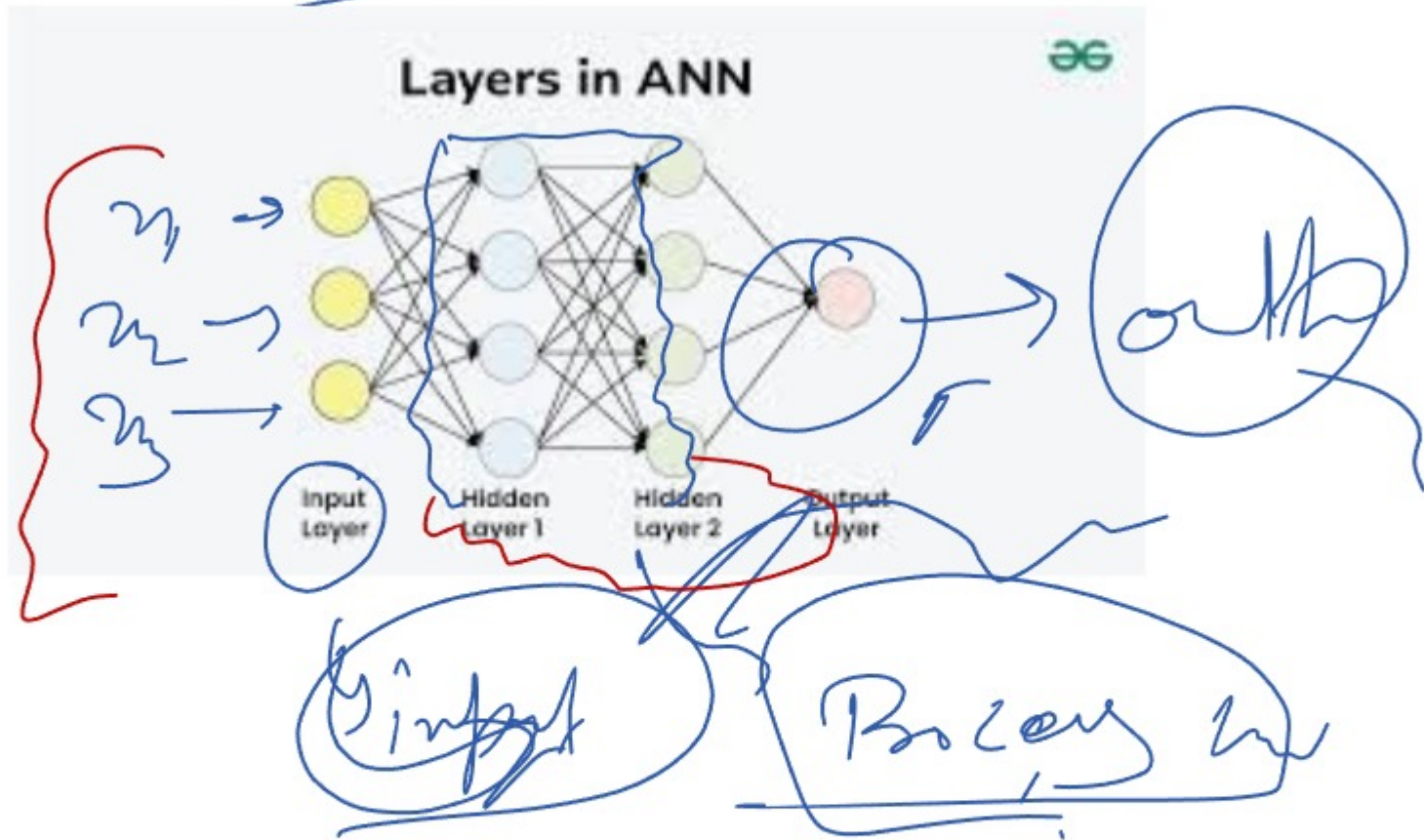


Image
Video
Animation

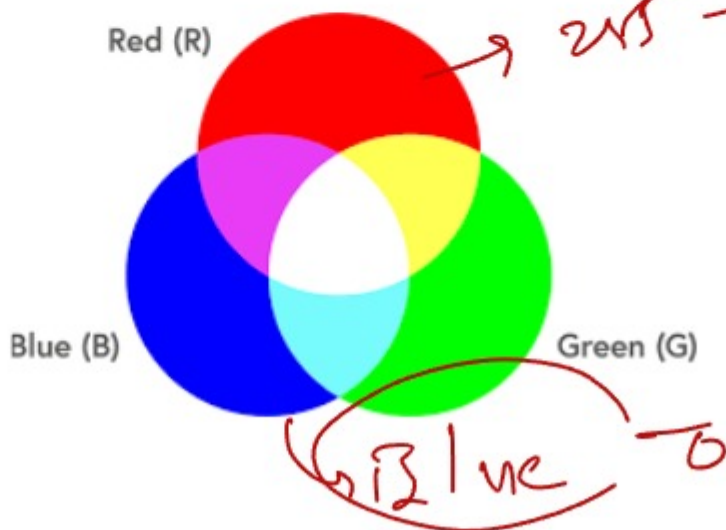
GPU

Image

group of P: 2

RGB
R → Red
G → Green
B → Blue

RGB (0 to 255)
Black is (0, 0, 0) [-1 1]



Black
→



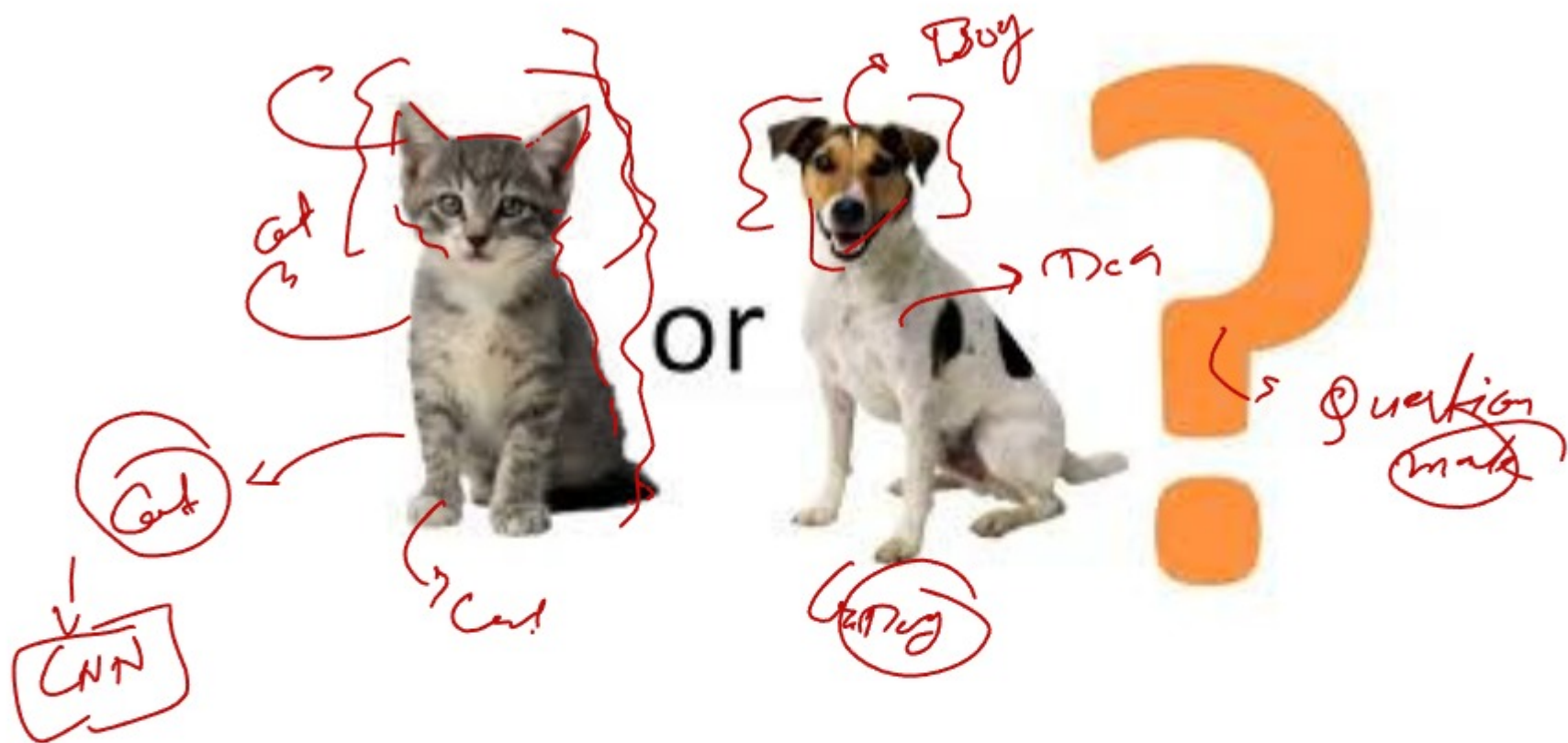
→ +1 white

3x3

+1	+1	-1
+1	-1	+1
-1	+1	+1

⇒





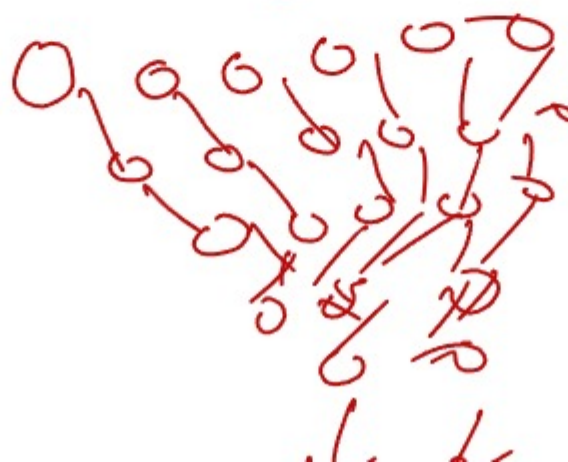
CNN Layer

- ① Convolutional layer $\Rightarrow$
- ② Pooling layer $\Rightarrow$
- ③ Flatten layer



Number $\Rightarrow$ Flatten()
Reveal

Printed 10d $\Rightarrow$ 10



Sequence

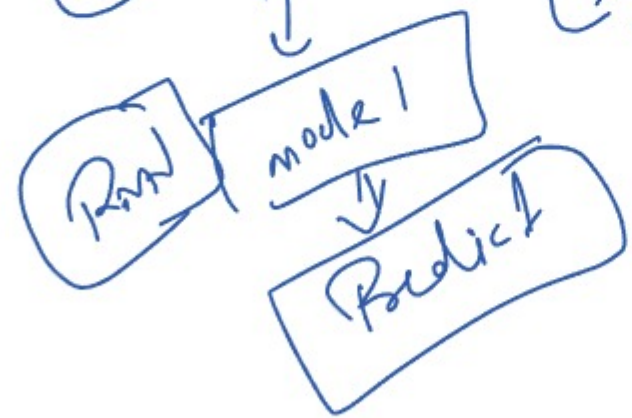
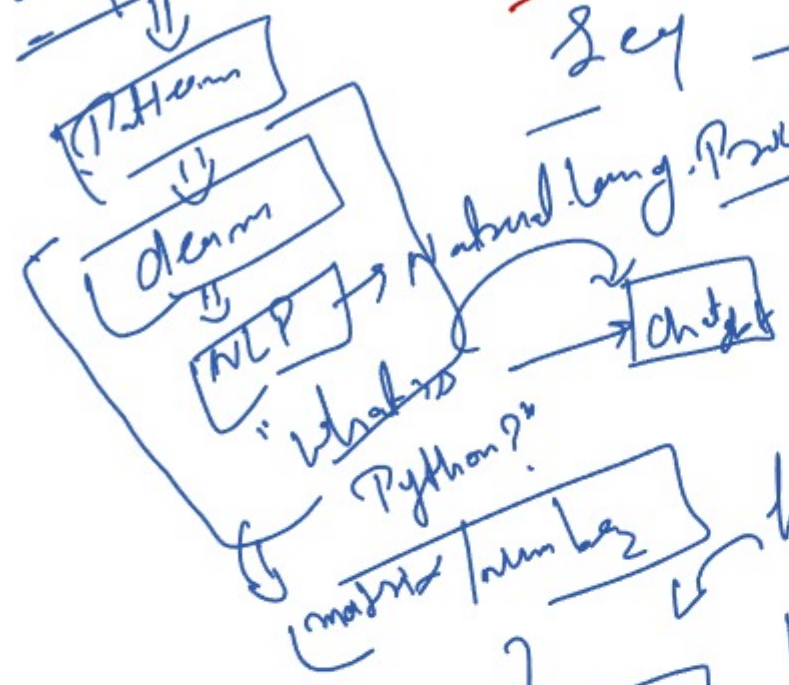
RNN - (Recurrent neural networks)

Seq -
Natural Lang. Processing

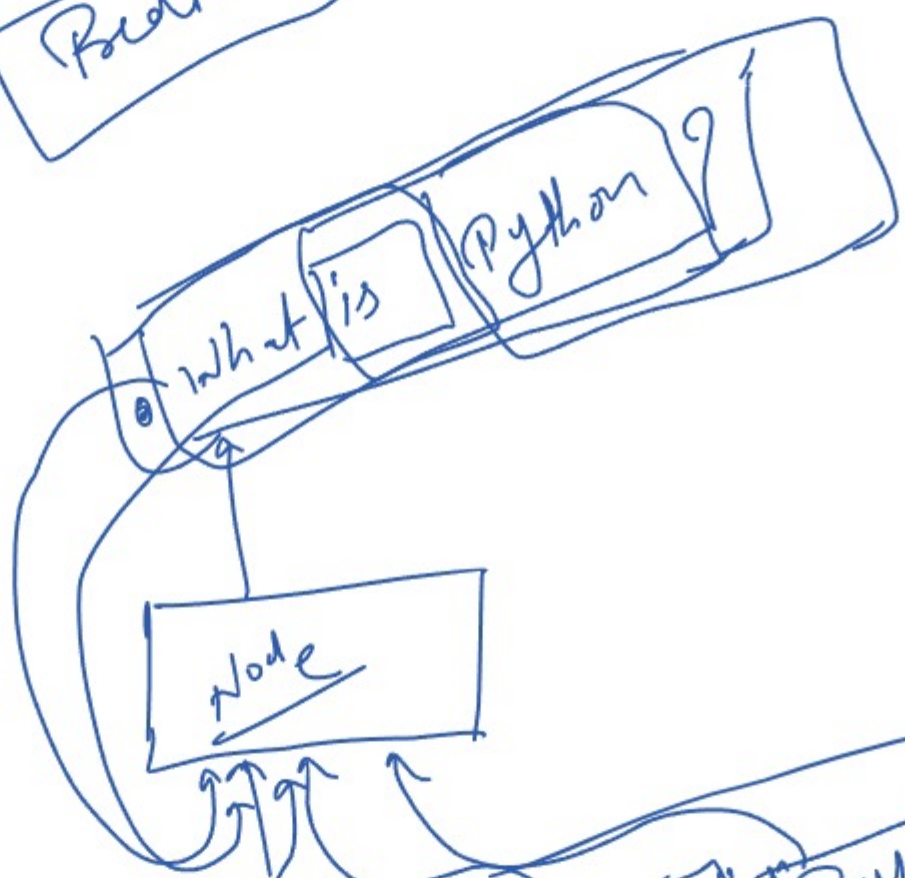
AI

NLP

Python is a list



RNN

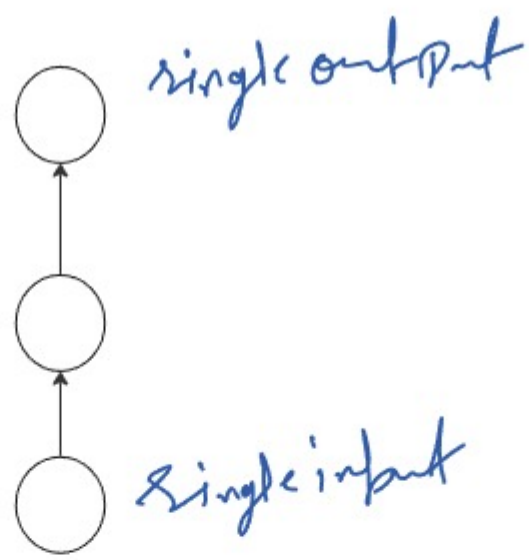


①

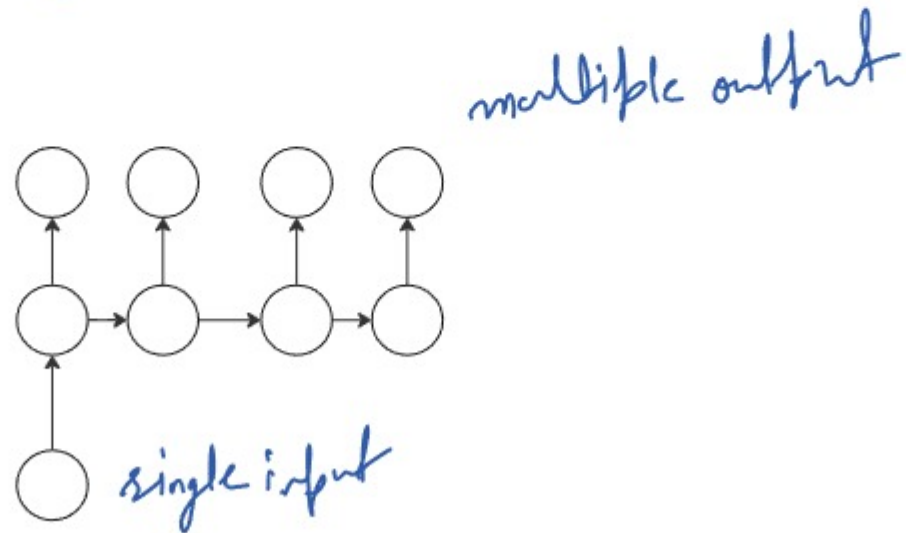
Types of RNN

- (i) one to one RNN
- (ii) one to many
- (iii) many to one
- (iv) many to many
- (v) LSTM
- (vi) GRU

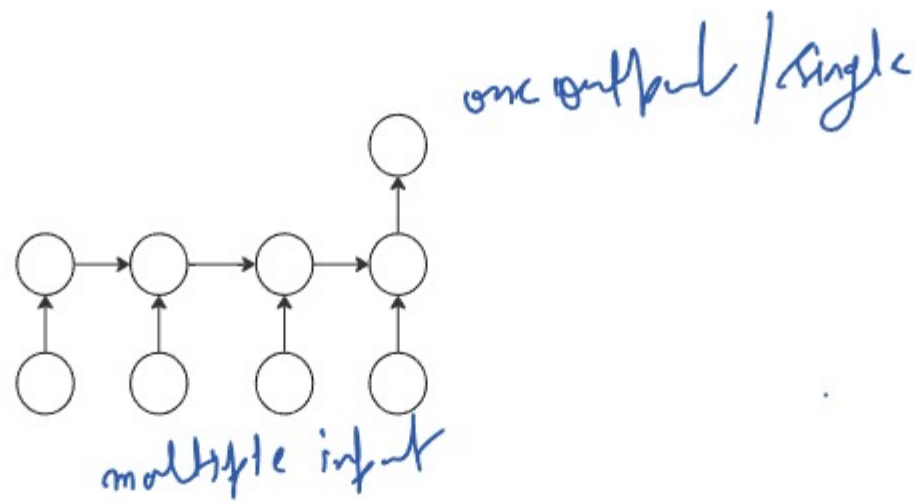
① one to one RNN



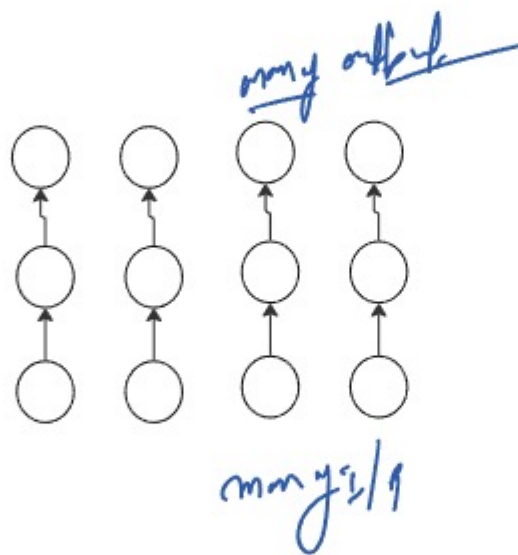
② one to many RNN



③ Many to one RNN



④ many to many RNN



LSTM

[Long short term memory]

[RNN]

[Vanishing Gradient Problem]

Advanced of RNN

Application of LSTM

(i) lang. trans

(ii) speech Rec

LSTM Architecture

(i) Input gate → Add

(ii) forget gate → Remove

(iii) output gate → output producing

GRU (Gated Recurrent Units)

faster and simpler than LSTM

Trans time ↓

Gradient Problem

GRU Archi

(i) update gate

(ii) Reset gate

Input →

Update gate
Reset gate

→ output

NLP
(Natural lang. Processing)
subset of AI
text classification
POS

NLP →

Text preprocessing steps

- (i) Lower casing
- (ii) LfMC remove
- # URC
- # @ # \$ #
- # Emoji

Text vectorization

(Text convert into number)

- (i) OHE
- (ii) Bow (Bag of words)
- (iii) N-grams
- (iv) TFIDF
- (v) Custom feature

Bag of Words

Aggr two classes are same to VC number same
unique

- # order are not matter
- # context do not matter